

Facultad de ingeniería

UNIVERSIDAD NACIONAL DE COLOMBIA



ParCheck

Ingeniería de Software I

Entrega 05

Tests unitarios

Informe de Análisis Estático de Código

Omar Alejandro Blanco Pineda

Sara Isabel Sepulveda Perez

Juan Esteban Ocampo Vidal

Samuel Mejia Ortiz

Daniel Felipe Orejuela Guzman

junio de 2026

1. Herramienta utilizada

La herramienta empleada para el análisis estático del código fue Pylint, un analizador de código fuente para Python que permite detectar errores, incumplimientos de estándares de estilo, problemas de diseño y posibles defectos de programación.

El análisis se ejecutó dentro de un contenedor Docker mediante el comando:

```
docker compose exec backend pylint apps parcheck_backend
```

Este comando analizó todas las aplicaciones Django contenidas en el directorio apps y la configuración principal del proyecto ubicada en parcheck_backend.

2. Configuración aplicada

Se utilizó la configuración predeterminada de Pylint.

No se evidenció el uso de un archivo de configuración personalizado (`.pylintrc`) ni la desactivación manual de reglas específicas durante la ejecución.

Por defecto, Pylint aplicó reglas relacionadas con:

- Estilo de código (PEP 8).
- Documentación de módulos, clases y métodos.
- Organización de imports.
- Variables no utilizadas.
- Posibles errores de programación.
- Calidad de diseño orientado a objetos.

3. Resultados obtenidos

El análisis detectó una gran cantidad de advertencias y errores distribuidos en las aplicaciones `eventos`, `finanzas`, `parches`, `users` y en la configuración principal del proyecto.

3.1 Problemas de documentación

La mayoría de los archivos carecen de documentación básica.

Ejemplos:

- `missing-module-docstring` (C0114)
- `missing-class-docstring` (C0115)
- `missing-function-docstring` (C0116)

Estos mensajes indican que módulos, clases y funciones no poseen comentarios descriptivos que faciliten el mantenimiento y comprensión del código.

3.2 Problemas de formato y estilo

Se detectaron múltiples incumplimientos de las convenciones de estilo:

- Líneas demasiado largas (`line-too-long`).
- Espacios en blanco innecesarios (`trailing-whitespace`).
- Ausencia de salto de línea al final del archivo (`missing-final-newline`).
- Orden incorrecto de imports (`wrong-import-order`).
- Imports fuera de la cabecera del archivo (`import-outside-toplevel`).

Estos problemas afectan principalmente la legibilidad y mantenibilidad del código.

3.3 Imports y variables sin uso

Se encontraron varios imports que no son utilizados:

- `unused-import` (W0611)
- `unused-argument` (W0613)

Por ejemplo:

- `django.contrib.admin`
- `django.test.TestCase`
- `psycpg2`

Eliminar código no utilizado ayuda a reducir complejidad y mejorar claridad.

3.4 Errores potenciales

Pylint reportó varios errores de tipo:

- `no-member` (E1101)
- `invalid-str-returned` (E0307)

Ejemplos:

- Acceso a atributos inexistentes como `objects` o `DoesNotExist`.
- Métodos `____str____()` que aparentemente no retornan un valor de tipo cadena.

En proyectos Django algunos mensajes `no-member` pueden ser falsos positivos debido a la generación dinámica de atributos por parte del ORM.

3.5 Problemas de diseño

Se identificaron advertencias relacionadas con diseño orientado a objetos:

- `too-few-public-methods (R0903)`

Esto ocurre cuando una clase posee muy pocos métodos públicos para justificar su existencia como clase independiente.

3.6 Problemas en migraciones

Las migraciones generadas automáticamente por Django presentan advertencias como:

- `invalid-name`
- `line-too-long`
- `missing-module-docstring`

Estos mensajes suelen ser normales en archivos autogenerados y generalmente no requieren corrección manual.

4. Conclusiones

El análisis realizado con Pylint permitió identificar problemas de documentación, estilo, organización de imports y posibles errores de programación.

Los problemas más frecuentes corresponden a:

1. Falta de documentación en módulos, clases y métodos.
2. Incumplimiento de reglas de formato y estilo.
3. Imports y variables no utilizadas.
4. Advertencias relacionadas con el ORM de Django.
5. Archivos de migración que generan advertencias automáticas.

Se recomienda priorizar la corrección de los errores de tipo (`E1101`, `E0307`) y posteriormente mejorar la documentación y el cumplimiento de las convenciones de estilo para incrementar la mantenibilidad y calidad general del proyecto.

Evidencia de Ejecución

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python ⚠ + - [ ] [ ] ... | [ ] [ ] X

==apps.eventos.migrations.0001_initial:[5:14]
==apps.finanzas.migrations.0001_initial:[5:14]
class Migration(migrations.Migration):

    initial = True

    dependencies = [
    ]

    operations = [
        migrations.CreateModel( (duplicate-code)
        )
    ]

-----
Your code has been rated at 1.95/10
```

Evidencia de Corrección

```
==apps.eventos.migrations.0001_initial:[5:14]
==apps.finanzas.migrations.0001_initial:[5:14]
class Migration(migrations.Migration):

    initial = True

    dependencies = [
    ]

    operations = [
        migrations.CreateModel( (duplicate-code)
        )
    ]

-----
Your code has been rated at 2.15/10 (previous run: 1.95/10, +0.20)
```